

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO



FACULTAD DE ESTUDIOS SUPERIORES ARAGÓN

Inteligencia Artificial

Grupo: 1957

Proyecto A: A Estrella

equipo:

- RODRIGUEZ FRANCO JESUS CHARBEL
- Tellez Lopez Angel

6 DE NOVIEMBRE DE 2026

Contenido

1. Qué hace el programa 2
1.1 Reglas del recorrido 2
1.2 Cómo se ejecuta 2
2. Conceptos que necesitas antes de leer el código 4
2.1 La idea de A* 4
2.2 Los tres números de cada casilla: g, h y f 4
2.3 La heurística: distancia Manhattan 4
2.4 Lista abierta y lista cerrada 4
2.5 El «padre» de una casilla 5
3. Cómo está organizado el archivo 6
3.1 Constantes 6
3.2 Atributos de la partida 6
3.3 La clase Nodo 7
3.4 Quién llama a quién 7
4. Las funciones, una por una 8
4.1 Entrada de datos 8
4.2 Validación de casillas 11
4.3 Costo y heurística 13
4.4 El algoritmo A* 14
4.5 Mostrar resultados 19
4.6 main: el punto de entrada 22
5. Ejemplo completo: de (1, 1) a (10, 10) 24
5.1 Las primeras vueltas de A* 24
5.2 Resultado 24
6. Pruebas automáticas 26
7. Observaciones 27

1. Qué hace el programa

`Laberinto.java` resuelve un laberinto de 10 × 10 casillas con el algoritmo A* (se lee «A estrella»). El usuario elige una casilla de inicio y una de meta, y el programa encuentra el camino **más barato** entre ambas, lo dibuja sobre el tablero y muestra cuántos pasos tiene y cuánto cuesta.

Este es el laberinto. Las casillas con **X** son obstáculos (no se puede pasar por ellas). Las casillas vacías cuestan 1 y las que tienen un número cuestan ese número.

	1	2	3	4	5	6	7	8	9	10
1			4		2			X		
2	4		X		3	X	2			X
3	X		X	2		4	3	X		
4	3	3	X			3			2	X
5		5	3		X		X	X		
6			X	X	2	X		4		X
7	X	X		X		5	2			
8		3		2		3	2		3	X
9			X	X	X		X	X		
10			3		3				2	

1.1 Reglas del recorrido

- **Movimientos permitidos:** arriba, abajo, izquierda y derecha. No hay movimientos en diagonal.
- **Costo:** se paga al **entrar** a una casilla. Por ejemplo, entrar a una casilla con un 4 suma 4 al costo total.
- **La casilla de inicio no suma costo**, porque ya estás en ella.
- **Obstáculos:** las casillas X no se pueden pisar nunca.

1.2 Cómo se ejecuta

Se necesita Java 11 o superior (puedes comprobarlo con `javac -version`) y una terminal que soporte colores ANSI: la de VS Code, Windows Terminal o cualquier terminal de Mac o Linux. Para descargar el proyecto:

```
git clone https://github.com/jesus24e/proyecto_A_estrella.git
cd proyecto_A_estrella
```

Después, desde la carpeta que contiene Laberinto.java:

```
java Laberinto.java
```

El programa muestra el tablero y pide cuatro números, siempre del 1 al 10:

```
Fila inicio (y): 1
Columna inicio (x): 1
Fila fin (y): 10
Columna fin (x): 10
```

Si escribes algo que no es un número, una casilla fuera del tablero, un obstáculo, o pones la meta en el mismo lugar que el inicio, el programa te avisa y vuelve a preguntar.

Con las cuatro coordenadas, el programa muestra el tablero con la ruta encontrada, la lista de casillas del camino de inicio a meta, el número de pasos y el costo total. Si no existe ninguna ruta entre las dos casillas, lo indica. En el tablero, los colores significan:

Símbolo	Color	Significado
`P`	Azul (fondo)	Casilla de inicio
`M`	Morado	Meta
`X`	Rojo	Obstáculo
`Carácter de la casilla`	Azul (fondo)	Casilla que forma parte de la ruta

Filas y columnas: y = fila, x = columna

En todo el código, **y** es la fila (de arriba a abajo) y **x** es la columna (de izquierda a derecha).

El usuario escribe números del **1 al 10**, pero internamente Java numera los arreglos del **0 al 9**. Por eso el programa resta 1 al leer y suma 1 al mostrar.

2. Conceptos que necesitas antes de leer el código

2.1 La idea de A*

Buscar un camino «a lo bruto» significa probar todas las rutas posibles, lo cual es lento. A* es más listo: en cada momento decide qué casilla explorar a continuación usando dos datos:

- **Lo que ya me costó llegar hasta aquí** (esto se conoce con exactitud).
- **Lo que calculo que me falta para llegar a la meta** (esto es una estimación).

Sumando ambos obtiene una puntuación. Siempre explora primero la casilla con la puntuación más baja, es decir, la que parece formar parte del camino más barato.

2.2 Los tres números de cada casilla: g, h y f

Valor	Nombre	Qué significa
`g`	Costo real	Lo que cuesta llegar desde el inicio hasta esta casilla por el mejor camino encontrado hasta ahora.
`h`	Heurística	Una estimación de lo que falta desde esta casilla hasta la meta.
`f`	Puntuación	`f = g + h`. Es el costo estimado de un camino completo que pase por esta casilla. A* siempre elige la casilla con menor `f`.

2.3 La heurística: distancia Manhattan

Para estimar lo que falta, el programa usa la **distancia Manhattan**: cuántas filas y cuántas columnas te separan de la meta, sumadas.

```
h = |fila - fila_meta| + |columna - columna_meta|
```

Ejemplo: si estás en la fila 1, columna 1 y la meta está en la fila 10, columna 10, entonces h = 9 + 9 = 18.

> **¿Por qué esta estimación garantiza el camino más barato?**
>
> Cada paso cuesta **al menos 1** y cada paso te acerca a la meta **como mucho 1** casilla. Por eso la distancia Manhattan **nunca exagera** lo que falta: el costo real siempre es igual o mayor.
>
> Una heurística que nunca exagera se llama **admisible**, y con ella A* garantiza encontrar la ruta de menor costo. Las pruebas del proyecto lo confirmaron comparando A* contra otro algoritmo en los 5550 pares posibles de inicio y meta.

2.4 Lista abierta y lista cerrada

A* lleva dos listas de casillas:

- **Lista abierta:** casillas que ya descubrió pero todavía no exploró. Son los «candidatos».
- **Lista cerrada:** casillas que ya exploró. No se vuelven a revisar.

Al principio, la lista abierta solo tiene la casilla de inicio. En cada vuelta, A* saca de la lista abierta la casilla con menor `f`, la pasa a la cerrada, y agrega a la abierta sus vecinas. Termina cuando saca la meta (encontró el camino) o cuando la lista abierta queda vacía (no hay camino).

2.5 El «padre» de una casilla

Cada vez que A* llega a una casilla, apunta desde qué casilla vino. A esa casilla anterior se le llama **padre**. Al llegar a la meta, basta con ir siguiendo los padres hacia atrás (meta → padre → padre del padre → … → inicio) para reconstruir el camino completo.

3. Cómo está organizado el archivo

Todo el programa está en una sola clase, `Laberinto`. Dentro tiene, en este orden: las constantes, los atributos (variables de la partida), una clase interna llamada `Nodo`, y las funciones.

3.1 Constantes

Son valores fijos que no cambian durante la ejecución (`static final`).

```
static final char[][] MAPA = {
    {'.', '.', '4', '.', '2', '.', '.', 'X', '.', '.'},
    {'4', '.', 'X', '.', '3', 'X', '2', '.', '.', 'X'},
    ...
};
```

- **`MAPA`**: matriz de caracteres de 10 × 10 con el laberinto. `'.'` significa costo 1, un dígito (`'2'` … `'5'`) es el costo de esa casilla y `'X'` es un obstáculo. Se accede como `MAPA[fila][columna]`.
- **`PLAYER`** (`'P'`), **`META`** (`'M'`) y **`OBSTACULO`** (`'X'`): los caracteres que se usan para dibujar y comparar.
- **`COLOR_JUGADOR`**, **`COLOR_OBSTACULO`**, **`COLOR_META`** y **`COLOR_RESET`**: códigos de color ANSI.

> **¿Qué es un código ANSI?**
> Es una secuencia especial de texto, como `\u001B[1;31m`, que la terminal no imprime sino que interpreta como una orden: «a partir de aquí escribe en rojo». `COLOR_RESET` vuelve al color normal.
>
> Por eso cada casilla de color se imprime como: color + carácter + reset. Si la terminal no soporta ANSI, verás esos códigos como texto raro.

3.2 Atributos de la partida

```
int player_xpos = -1;
int player_ypos = -1;
int meta_xpos = -1;
int meta_ypos = -1;
ArrayList<int[]> movimientos = new ArrayList<>();
```

Atributo	Qué guarda
`player_ypos`, `player_xpos`	Fila y columna de la casilla de inicio (de 0 a 9). Empiezan en -1, que significa «todavía no definido».
`meta_ypos`, `meta_xpos`	Fila y columna de la meta (de 0 a 9). También empiezan en -1.
`movimientos`	Lista de casillas que se pintan como ruta en el tablero. Cada casilla es un arreglo `{fila, columna}`.

3.3 La clase Nodo

Un `Nodo` representa una casilla **durante la búsqueda**. No es la casilla del mapa en sí, sino la información que A* necesita recordar sobre ella.

```
static class Nodo {
    int y;        // fila
    int x;        // columna
    int g;        // costo real desde el inicio
    int h;        // estimacion hasta la meta
    int f;        // g + h
    Nodo padre;   // casilla desde la que se llego

    Nodo(int y, int x, int g, int h, Nodo padre){
        this.y = y;
        this.x = x;
        this.g = g;
        this.h = h;
        this.f = g+h;
        this.padre = padre;
    }
}
```

El constructor recibe la posición, `g`, `h` y el padre, y calcula `f` automáticamente. El nodo de inicio tiene `padre = null` porque no viene de ninguna parte.

3.4 Quién llama a quién

Este esquema muestra cómo se conectan las funciones. Cada nivel de sangría indica que la función de arriba llama a las de abajo.

```
main
├── imprimirTablero
│   └── casillaEnMovimientos
├── leerEntero
├── definirInicio
│   └── comprobarValidezCasilla
│       └── esTransitable
│           └── dentroDelTablero
├── guardarMovimiento
├── definirFin
│   └── comprobarValidezCasilla  (igual que arriba)
└── mostrarSolucion
    ├── resolverAEstrella
    │   ├── heuristica
    │   ├── buscarMenorF
    │   ├── revisarVecinos
    │   │   ├── esTransitable
    │   │   ├── buscarNodo
    │   │   ├── costo
    │   │   └── heuristica
    │   └── reconstruirRuta
    ├── imprimirTablero
    └── costo
```

4. Las funciones, una por una

Las funciones están agrupadas según su papel: leer datos, validar casillas, calcular costos, ejecutar A* y mostrar resultados. Al final está `main`, que las une todas.

Para cada función hay una ficha con su firma, qué recibe, qué devuelve y quién la usa; luego el código y la explicación paso a paso.

4.1 Entrada de datos

`leerEntero`

Dato	Detalle
Firma	`static int leerEntero(Scanner teclado, String mensaje)`
Parámetros	`teclado`: el lector de la entrada estándar. `mensaje`: el texto que se muestra al pedir el número.
Devuelve	El número entero que escribió el usuario.
La llama	`main`, cuatro veces (fila y columna del inicio y de la meta).

```
static int leerEntero(Scanner teclado, String mensaje){
    System.out.print(mensaje);
    while (!teclado.hasNextInt()) {
        System.out.println("Eso no es un numero.");
        teclado.next();
        System.out.print(mensaje);
    }
    return teclado.nextInt();
}
```

Paso a paso

1. Muestra el mensaje, por ejemplo `Fila inicio (y): ` .
2. `hasNextInt()` mira lo próximo que escribió el usuario **sin consumirlo** y responde si es un entero.
3. Mientras no lo sea:
   a) imprime «Eso no es un numero.»,
   b) `teclado.next()` descarta esa palabra (si no se descartara, el bucle la vería una y otra vez y no terminaría nunca),
   c) vuelve a mostrar el mensaje.
4. Cuando por fin hay un entero, `nextInt()` lo lee y lo devuelve.

Esta función **solo comprueba que sea un número**, no que esté entre 1 y 10. De eso se encargan `definirInicio` y `definirFin`.

definirInicio

Dato	Detalle
Firma	`boolean definirInicio(int inicio_y, int inicio_x)`
Parámetros	Fila y columna del inicio tal como las escribió el usuario (de 1 a 10).
Devuelve	`true` si la casilla es válida y se guardó; `false` si no.
La llama	`main`.

```
boolean definirInicio(int inicio_y,int inicio_x){
    int columna_inicio = inicio_x-1;
    int fila_inicio = inicio_y-1;
    boolean validez = false;

    if (comprobarValidezCasilla(fila_inicio, columna_inicio)) {
        validez = true;
        player_xpos = columna_inicio;
        player_ypos = fila_inicio;
    }

    return validez;
}
```

Paso a paso

1. Resta 1 a la fila y a la columna para pasar de la numeración del usuario (1 a 10) a la de Java (0 a 9).
2. Empieza suponiendo que la casilla no es válida (`validez = false`).
3. Pregunta a `comprobarValidezCasilla` si la casilla existe y no es obstáculo. Si la respuesta es no, esa función ya imprimió el mensaje de error.
4. Si es válida, guarda la posición en `player_ypos` y `player_xpos` y marca `validez = true`.
5. Devuelve `validez`. Con eso `main` sabe si tiene que volver a preguntar.

Ejemplo

El usuario escribe fila 1, columna 8. Se convierte a `(0, 7)`. En el mapa `MAPA[0][7]` es `'X'`, así que se imprime el error y se devuelve `false`.

definirFin

Dato	Detalle
Firma	`boolean definirFin(int fin_y, int fin_x)`
Parámetros	Fila y columna de la meta tal como las escribió el usuario (de 1 a 10).
Devuelve	`true` si la meta es válida y se guardó; `false` si no.
La llama	`main`, después de definir el inicio.

```
boolean definirFin(int fin_y, int fin_x){
    int columna_fin = fin_x-1;
    int fila_fin = fin_y-1;
    boolean validez = false;

    if (comprobarValidezCasilla(fila_fin, columna_fin)) {
        if (fila_fin == player_ypos && columna_fin == player_xpos) {
            System.out.println("la meta no puede estar en la casilla de
inicio, ...");
        }else{
            validez = true;
            meta_xpos = columna_fin;
            meta_ypos = fila_fin;
        }
    }

    return validez;
}
```

Paso a paso

1. Convierte las coordenadas a índices de 0 a 9, igual que `definirInicio`.
2. Comprueba que la casilla exista y no sea obstáculo.
3. Además comprueba que **no sea la misma casilla que el inicio**. Si lo es, avisa y devuelve `false`.
4. Si todo está bien, guarda la meta en `meta_ypos` y `meta_xpos` y devuelve `true`.

Es casi igual a `definirInicio`; la única diferencia es la comprobación extra de que la meta no coincida con el inicio.

`guardarMovimiento`

Dato	Detalle
Firma	`void guardarMovimiento(int y, int x)`
Parámetros	Fila y columna (de 0 a 9).
Devuelve	Nada.
La llama	`main`, justo después de aceptar el inicio.

```
void guardarMovimiento(int y,int x){
    movimientos.add(new int[]{y,x});
}
```

Agrega la casilla `{y, x}` a la lista `movimientos`. Se usa para registrar el inicio. Más adelante, cuando A* encuentra la ruta, `mostrarSolucion` reemplaza la lista entera por la ruta completa.

4.2 Validación de casillas

Hay tres funciones de validación y cada una se apoya en la anterior:

```
comprobarValidezCasilla  →  esTransitable  →  dentroDelTablero
```

`dentroDelTablero`

Dato	Detalle
Firma	`boolean dentroDelTablero(int y, int x)`
Parámetros	Fila y columna (de 0 a 9, aunque puede recibir cualquier número).
Devuelve	`true` si la casilla está dentro del mapa.
La llama	`esTransitable`.

```
boolean dentroDelTablero(int y, int x){
    boolean validez = true;

    if ((y<0 || x<0)||(y>MAPA.length-1 || x>MAPA[0].length-1)) {
        validez = false;
    }

    return validez;
}
```

Paso a paso

1. Supone que la casilla es válida.
2. Si la fila o la columna es negativa, o mayor que el último índice (`MAPA.length-1` es 9 para las filas y `MAPA[0].length-1` es 9 para las columnas), la marca como no válida.
3. Devuelve el resultado.

Esta comprobación es imprescindible: si se intentara leer `MAPA[-1][3]` o `MAPA[10][3]`, Java lanzaría un error (`ArrayIndexOutOfBoundsException`) y el programa se cerraría. Eso pasaría, por ejemplo, al revisar las vecinas de una casilla del borde.

`esTransitable`

Dato	Detalle
Firma	`boolean esTransitable(int y, int x)`
Parámetros	Fila y columna (de 0 a 9).
Devuelve	`true` si la casilla está dentro del mapa y no es un obstáculo.
La llama	`comprobarValidezCasilla` y `revisarVecinos`.

```
boolean esTransitable(int y, int x){
    boolean validez = false;

    if (dentroDelTablero(y, x)) {
        if (MAPA[y][x]!=OBSTACULO) {
            validez = true;
        }
    }

    return validez;
}
```

Paso a paso

1. Supone que no se puede pisar.
2. Primero comprueba que esté dentro del tablero. El orden importa: si no lo está, ni siquiera mira el mapa, así evita el error de índice fuera de rango.
3. Si está dentro y el carácter no es `'X'`, se puede pisar.

No imprime nada. Esto es intencional: A* la llama muchísimas veces mientras busca, y si imprimiera mensajes llenaría la terminal.

`comprobarValidezCasilla`

Dato	Detalle
Firma	`boolean comprobarValidezCasilla(int y, int x)`
Parámetros	Fila y columna (de 0 a 9).
Devuelve	Lo mismo que `esTransitable`.
La llama	`definirInicio` y `definirFin`.

```
boolean comprobarValidezCasilla(int y, int x){
    boolean validez = false;

    if (!esTransitable(y, x)) {
        System.out.println("error, la posicion esta fuera del rango ...");
    }else{
        validez = true;
    }

    return validez;
}
```

Es la versión «para el usuario» de `esTransitable`: hace la misma comprobación, pero si la casilla no vale, **imprime un mensaje de error** para que el usuario sepa por qué se le vuelve a preguntar.

4.3 Costo y heurística

`costo`

Dato	Detalle
Firma	`int costo(int y, int x)`
Parámetros	Fila y columna de una casilla transitable.
Devuelve	Lo que cuesta entrar a esa casilla.
La llama	`revisarVecinos` y `mostrarSolucion`.

```
int costo(int y, int x){
    int valor = 1;

    if (MAPA[y][x]!='.') {
        valor = Character.getNumericValue(MAPA[y][x]);
    }

    return valor;
}
```

Paso a paso

1. Supone que el costo es 1 (el de una casilla vacía `'.'`).
2. Si la casilla tiene otro carácter, es un dígito; `Character.getNumericValue` convierte el carácter `'4'` en el número `4`.
3. Devuelve el costo.

> **Detalle**
> Un `char` como `'4'` no es el número 4: internamente es el código 52. Por eso hace falta `getNumericValue` para convertirlo. La función nunca se llama con una `'X'`, porque antes siempre se comprueba que la casilla sea transitable.

`heuristica`

Dato	Detalle
Firma	`int heuristica(int y, int x)`
Parámetros	Fila y columna de una casilla.
Devuelve	La distancia Manhattan desde esa casilla hasta la meta.
La llama	`resolverAEstrella` (para el nodo inicial) y `revisarVecinos` (para cada nodo nuevo).

```
int heuristica(int y, int x){
    int distancia_y = Math.abs(y - meta_ypos);
    int distancia_x = Math.abs(x - meta_xpos);

    return distancia_y + distancia_x;
}
```

Paso a paso

1. Calcula cuántas filas hay entre la casilla y la meta. `Math.abs` quita el signo, porque da igual si la meta está arriba o abajo.
2. Calcula cuántas columnas hay entre la casilla y la meta.
3. Devuelve la suma.

Es la estimación h explicada en la sección 2.3. Ignora los obstáculos y los costos: solo mide la distancia, como si el camino estuviera libre.

4.4 El algoritmo A*

Estas cinco funciones forman el corazón del programa. `resolverAEstrella` es la principal y usa las otras cuatro como ayudantes.

`buscarNodo`

Dato	Detalle
Firma	`Nodo buscarNodo(ArrayList<Nodo> lista, int y, int x)`
Parámetros	Una lista de nodos (la abierta o la cerrada) y una posición.
Devuelve	El nodo de esa posición si está en la lista, o `null` si no está.
La llama	`revisarVecinos`.

```
Nodo buscarNodo(ArrayList<Nodo> lista, int y, int x){
    Nodo encontrado = null;

    for(Nodo nodo: lista){
        if (nodo.y == y && nodo.x == x) {
            encontrado = nodo;
        }
    }

    return encontrado;
}
```

Recorre la lista nodo por nodo y, si alguno tiene la misma fila y columna, lo guarda. Devolver `null` es la forma de decir «no está». Se usa para saber si una casilla ya fue explorada (lista cerrada) o ya es candidata (lista abierta).

buscarMenorF

Dato	Detalle
Firma	`Nodo buscarMenorF(ArrayList<Nodo> lista)`
Parámetros	La lista abierta (nunca vacía cuando se llama).
Devuelve	El nodo con menor `f`.
La llama	`resolverAEstrella`.

```
Nodo buscarMenorF(ArrayList<Nodo> lista){
    Nodo menor = lista.get(0);

    for(Nodo nodo: lista){
        if (nodo.f < menor.f) {
            menor = nodo;
        }
    }

    return menor;
}
```

Paso a paso

1. Toma el primer nodo como «el menor hasta ahora».
2. Recorre la lista; cada vez que encuentra uno con `f` estrictamente menor, lo pasa a ser el menor.
3. Devuelve el menor.

Si hay empate, gana el que aparece primero en la lista, porque la comparación es `<` y no `<=`. Esto puede cambiar **cuál** de dos rutas igual de baratas se elige, pero nunca el costo.

revisarVecinos

Dato	Detalle
Firma	`void revisarVecinos(Nodo actual, ArrayList<Nodo> lista_abierta, ArrayList<Nodo> lista_cerrada)`
Parámetros	El nodo que se está explorando y las dos listas.
Devuelve	Nada; modifica la lista abierta.
La llama	`resolverAEstrella`, una vez por cada nodo explorado.

```
void revisarVecinos(Nodo actual, ArrayList<Nodo> lista_abierta, ArrayList<Nodo>
lista_cerrada){
    int[] vecinos_y = {actual.y-1, actual.y+1, actual.y, actual.y};
    int[] vecinos_x = {actual.x, actual.x, actual.x-1, actual.x+1};

    for(int i = 0; i < 4; i++){
        int y = vecinos_y[i];
        int x = vecinos_x[i];

        if (esTransitable(y, x) && buscarNodo(lista_cerrada, y, x) == null) {
            int nuevo_g = actual.g + costo(y, x);
            Nodo vecino = buscarNodo(lista_abierta, y, x);

            if (vecino == null) {
                lista_abierta.add(new Nodo(y, x, nuevo_g, heuristica(y, x),
actual));
            }else if(nuevo_g < vecino.g){
                vecino.g = nuevo_g;
                vecino.f = nuevo_g + vecino.h;
                vecino.padre = actual;
            }
        }
    }
}
```

Paso a paso

1. Prepara las posiciones de las 4 vecinas usando dos arreglos paralelos. La posición `i` de ambos forma una vecina:
   a) `i = 0`: arriba (fila - 1)
   b) `i = 1`: abajo (fila + 1)
   c) `i = 2`: izquierda (columna - 1)
   d) `i = 3`: derecha (columna + 1)
2. Para cada vecina, la **ignora** si no se puede pisar (fuera del tablero u obstáculo) o si ya está en la lista cerrada.
3. Calcula `nuevo_g`: lo que costaría llegar a la vecina pasando por el nodo actual. Es el `g` del actual más el costo de entrar a la vecina.
4. Busca si la vecina ya está en la lista abierta:
   a) **Si no está**, es una casilla recién descubierta: crea un nodo nuevo con `g = nuevo_g`, su heurística y `padre = actual`, y lo agrega a la lista abierta.
   b) **Si ya está** pero `nuevo_g` es menor que su `g`, acabamos de encontrar un camino más barato hacia ella: se actualizan su `g`, su `f` y su padre.
   c) Si ya está y `nuevo_g` no es menor, no se hace nada: el camino anterior era igual o mejor.

¿Por qué se actualiza el padre?

Porque el padre define el camino. Si llegar a una casilla desde el nodo actual es más barato que desde el padre anterior, la mejor ruta hacia ella pasa ahora por el nodo actual. Al cambiar el padre, la reconstrucción final seguirá ese camino más barato. Ojo: `h` no cambia, porque la distancia a la meta es la misma; por eso `f` se recalcula como `nuevo_g + vecino.h`.

`reconstruirRuta`

Dato	Detalle
Firma	`ArrayList<int[]> reconstruirRuta(Nodo nodo_meta)`
Parámetros	El nodo de la meta, una vez alcanzada.
Devuelve	La ruta como lista de `{fila, columna}`, ordenada del inicio a la meta.
La llama	`resolverAEstrella`.

```
ArrayList<int[]> reconstruirRuta(Nodo nodo_meta){
    ArrayList<int[]> ruta = new ArrayList<>();
    Nodo nodo = nodo_meta;

    while (nodo != null) {
        ruta.add(0, new int[]{nodo.y, nodo.x});
        nodo = nodo.padre;
    }

    return ruta;
}
```

Paso a paso

1. Empieza en el nodo de la meta.
2. Mientras haya nodo, agrega su posición **al principio** de la lista (`add(0, ...)`) y salta a su padre.
3. El nodo de inicio tiene `padre = null`, así que ahí termina el bucle.

Como se recorre de la meta hacia el inicio, insertar al principio deja la lista ya ordenada de inicio a meta, sin tener que invertirla después.

Ejemplo

Si la cadena de padres es M → C → B → A (inicio), la lista va quedando así: `[M]`, luego `[C, M]`, luego `[B, C, M]` y por último `[A, B, C, M]`.

`resolverAEstrella`

Dato	Detalle
Firma	`ArrayList<int[]> resolverAEstrella()`

Dato	Detalle
Parámetros	Ninguno: usa el inicio y la meta guardados en los atributos.
Devuelve	La ruta de inicio a meta, o una lista vacía si no existe ruta.
La llama	`mostrarSolucion`.

```
ArrayList<int[]> resolverAEstrella(){
    ArrayList<Nodo> lista_abierta = new ArrayList<>();
    ArrayList<Nodo> lista_cerrada = new ArrayList<>();
    ArrayList<int[]> ruta = new ArrayList<>();
    boolean encontrado = false;

    lista_abierta.add(new Nodo(player_ypos, player_xpos, 0,
                               heuristica(player_ypos, player_xpos), null));

    while (!lista_abierta.isEmpty() && !encontrado) {
        Nodo actual = buscarMenorF(lista_abierta);
        lista_abierta.remove(actual);
        lista_cerrada.add(actual);

        if (actual.y == meta_ypos && actual.x == meta_xpos) {
            encontrado = true;
            ruta = reconstruirRuta(actual);
        }else{
            revisarVecinos(actual, lista_abierta, lista_cerrada);
        }
    }

    return ruta;
}
```

Paso a paso

1. Crea la lista abierta, la lista cerrada y una ruta vacía.
2. Mete en la lista abierta el nodo de inicio: `g = 0` (no ha costado nada llegar), `h` = distancia a la meta, y sin padre.
3. Repite mientras queden candidatos y no se haya encontrado la meta:
   a) Elige el candidato con menor `f` (`buscarMenorF`).
   b) Lo saca de la lista abierta y lo pone en la cerrada.
   c) Si es la meta, reconstruye la ruta y marca `encontrado = true` para salir del bucle.
   d) Si no, revisa sus vecinas para añadir o mejorar candidatos.
4. Devuelve la ruta. Si la lista abierta se vació sin encontrar la meta, la ruta sigue vacía, lo que significa que no hay camino.

¿Por qué se comprueba la meta al sacarla y no al descubrirla?

Cuando la meta se descubre como vecina, quizá exista otro camino más barato hacia ella que aún no se ha explorado. Solo cuando la meta es el nodo con menor `f` de toda la lista abierta hay garantía de que no existe un camino mejor.

4.5 Mostrar resultados

`imprimirTablero`

Dato	Detalle
Firma	`void imprimirTablero()`
Parámetros	Ninguno.
Devuelve	Nada; imprime en la terminal.
La llama	`main` (al principio) y `mostrarSolucion` (con la ruta).

```
void imprimirTablero(){
    System.out.println();
    System.out.print("   ");
    for(int conteo = 1; conteo<=MAPA[0].length; conteo++){
        System.out.print(conteo+" ");
    }
    System.out.println();
    System.out.println("   "+"_".repeat(MAPA[0].length*2));
    for(int y=0; y < MAPA.length; y++){
        if (y>=9) {
            System.out.print(y+1+"|");
        }else{
            System.out.print(y+1+" |");
        }
        for(int x = 0; x < MAPA[0].length; x++){
            if (y == player_ypos && x == player_xpos) {
                System.out.print(COLOR_JUGADOR + PLAYER + " " + COLOR_RESET);
            }else if(y == meta_ypos && x == meta_xpos){
                System.out.print(COLOR_META + META + " " + COLOR_RESET);
            }else if(MAPA[y][x]==OBSTACULO){
                System.out.print(COLOR_OBSTACULO + MAPA[y][x] + " " + COLOR_RESET);
            }else if(casillaEnMovimientos(y, x)){
                System.out.print(COLOR_JUGADOR + MAPA[y][x] + " " + COLOR_RESET);
            }else{
                System.out.print(MAPA[y][x] + " ");
            }
        }
        System.out.println();
    }
    System.out.println();
}
```

Paso a paso

1. Imprime la cabecera con los números de columna del 1 al 10 y una línea de guiones bajos debajo.
2. Para cada fila, imprime su número seguido de `|`. La fila 10 tiene dos cifras, así que se le quita el espacio extra para que todo quede alineado.
3. Para cada casilla de la fila decide qué dibujar, en este orden de prioridad:
   a) si es el inicio: una `P` con fondo azul;
   b) si es la meta: una `M` morada;
   c) si es obstáculo: una `X` roja;
   d) si es parte de la ruta: su carácter con fondo azul;
   e) si no: su carácter normal (`.` o el número).

El orden de los `if` importa: el inicio y la meta también forman parte de la ruta, pero como se comprueban antes, se dibujan como `P` y `M` y no como casillas de ruta.

`casillaEnMovimientos`

Dato	Detalle
Firma	`boolean casillaEnMovimientos(int y, int x)`
Parámetros	Fila y columna.
Devuelve	`true` si la casilla está en la lista `movimientos`.
La llama	`imprimirTablero`.

```
boolean casillaEnMovimientos(int y, int x){
    for(int[] pos:movimientos){
        if(y == pos[0] && x == pos[1]){
            return  true;
        }
    }
    return false;
}
```

Recorre la lista de casillas de la ruta y devuelve `true` en cuanto encuentra una coincidencia. Si termina de recorrerla sin encontrarla, devuelve `false`. Se usa para saber qué casillas pintar.

`mostrarSolucion`

Dato	Detalle
Firma	`void mostrarSolucion()`
Parámetros	Ninguno.
Devuelve	Nada; imprime el resultado.
La llama	`main`, al final.

```
void mostrarSolucion(){
    ArrayList<int[]> ruta = resolverAEstrella();

    if (ruta.isEmpty()) {
        imprimirTablero();
        System.out.println("no existe una ruta entre el inicio y la meta.");
    }else{
        int costo_total = 0;

        movimientos = ruta;
        imprimirTablero();

        System.out.println("ruta encontrada con A*:");
        System.out.println();
        for(int i = 0; i < ruta.size(); i++){
            int[] pos = ruta.get(i);
            String costo = i>0 ? String.valueOf(costo(pos[0], pos[1])):"0";
            System.out.println("fila " + (pos[0]+1) + ", columna " + (pos[1]+1) +
", costo: " + costo);
            if (i > 0) {
                costo_total = costo_total + costo(pos[0], pos[1]);
            }
        }
        System.out.println();
        System.out.println("pasos: " + (ruta.size()-1));
        System.out.println("costo total: " + costo_total);
    }
}
```

Paso a paso

1. Ejecuta A* y obtiene la ruta.
2. Si la ruta está vacía, dibuja el tablero y avisa que no hay camino.
3. Si hay ruta:
   a) la guarda en `movimientos` para que `imprimirTablero` la pinte;
   b) dibuja el tablero;
   c) recorre la ruta e imprime cada casilla sumando 1 a sus coordenadas para mostrarlas como las ve el usuario;
   d) la primera casilla (`i = 0`) es el inicio: se muestra con costo 0 y no se suma;
   e) el resto suma su costo a `costo_total`.
4. Imprime el número de pasos y el costo total.

Pasos es el número de casillas menos 1: una ruta de 19 casillas tiene 18 movimientos. La expresión `i>0 ? ... : "0"` es un operador ternario: «si `i > 0`, usa el costo de la casilla; si no, usa "0"».

4.6 main: el punto de entrada

Dato	Detalle
Firma	`public static void main(String[] args)`
Parámetros	Argumentos de la línea de comandos (no se usan).
Devuelve	Nada.
La llama	Java, al ejecutar el programa.

```
public static void main(String[] args) {
    Laberinto partida = new Laberinto();
    Scanner teclado = new Scanner(System.in);
    boolean validez = false;

    partida.imprimirTablero();
    System.out.println("Introduce los numeros de fila (y) y columna (x) ...");
    System.out.println();

    while (!validez) {
        int inicio_y = leerEntero(teclado, "Fila inicio (y): ");
        int inicio_x = leerEntero(teclado, "Columna inicio (x): ");
        validez = partida.definirInicio(inicio_y, inicio_x);
        if (validez) {
            partida.guardarMovimiento(partida.player_ypos, partida.player_xpos);
        }
    }

    validez = false;

    while (!validez) {
        int fin_y = leerEntero(teclado, "Fila fin (y): ");
        int fin_x = leerEntero(teclado, "Columna fin (x): ");
        validez = partida.definirFin(fin_y, fin_x);
    }

    partida.mostrarSolucion();

    teclado.close();
}
```

Paso a paso

1. Crea un objeto `Laberinto` llamado `partida` (que guarda el inicio, la meta y los movimientos) y un `Scanner` para leer el teclado.
2. Dibuja el tablero vacío y explica qué debe introducir el usuario.
3. **Bucle del inicio:** pide fila y columna hasta que `definirInicio` devuelva `true`. Cuando lo hace, guarda el inicio en `movimientos`.
4. Reinicia `validez` a `false` para reutilizar la variable.
5. **Bucle de la meta:** pide fila y columna hasta que `definirFin` devuelva `true`.
6. Llama a `mostrarSolucion`, que ejecuta A* e imprime el resultado.

7. Cierra el `Scanner`.

`main` es `static`, es decir, pertenece a la clase y no a un objeto. Por eso necesita crear `partida` para poder usar las funciones que no son `static`, como `definirInicio` o `mostrarSolucion`. En cambio, `leerEntero` sí es `static` y se llama directamente.

5. Ejemplo completo: de (1, 1) a (10, 10)

Veamos qué hace el programa cuando el usuario pide ir de la esquina superior izquierda a la inferior derecha. Internamente el inicio es `(0, 0)` y la meta `(9, 9)`, pero aquí se usan las coordenadas del usuario (1 a 10).

5.1 Las primeras vueltas de A*

La siguiente tabla muestra, en cada vuelta, qué nodo se exploró y cómo quedó la lista abierta después de revisar sus vecinas. Los valores están tomados de una ejecución real del programa.

Vuelta	Nodo explorado	Lista abierta al terminar la vuelta
1	(1,1) g=0 h=18 f=18	(2,1) f=21 · (1,2) f=18
2	(1,2) g=1 h=17 f=18	(2,1) f=21 · (2,2) f=18 · (1,3) f=21
3	(2,2) g=2 h=16 f=18	(2,1) f=21 · (1,3) f=21 · (3,2) f=18
4	(3,2) g=3 h=15 f=18	(2,1) f=21 · (1,3) f=21 · (4,2) f=20
5	(4,2) g=6 h=14 f=20	(2,1) f=21 · (1,3) f=21 · (5,2) f=24 · (4,1) f=24

Qué se observa:

- **Vuelta 1:** desde el inicio solo hay dos vecinas transitables, (2,1) que cuesta 4 y (1,2) que cuesta 1. Arriba y a la izquierda está el borde del tablero.
- **Vueltas 2 a 4:** A* baja por la columna 2 porque las casillas cuestan 1 y la `f` se mantiene en 18, el mínimo posible.
- **Vuelta 5:** entrar a (4,2) cuesta 3, así que `f` sube a 20. Las siguientes opciones, (5,2) y (4,1), son caras (`f` = 24). A partir de aquí las candidatas de `f` = 21 que dejó atrás se vuelven más atractivas y A* empieza a probar el camino por arriba.

En total, A* explora **49 de las 75 casillas transitables** antes de sacar la meta de la lista abierta. La heurística le permite dejar sin explorar las zonas que claramente no llevan a un camino barato.

5.2 Resultado

La ruta encontrada (en azul, igual que la casilla de inicio) va por la fila de arriba, baja por la columna 9 y termina en la meta:

	1	2	3	4	5	6	7	8	9	10
1	P	·	4	·	2	·	·	X		
2	4		X		3	X	2	·	·	X
3	X		X	2		4	3	X	·	
4	3	3	X			3			2	X
5		5	3		X		X	X	·	
6			X	X	2	X		4	·	X
7	X	X		X		5	2		·	
8		3		2		3	2		3	X
9			X	X	X		X	X	·	·

10			3		3				2	**M**

#	Casilla (fila, columna)	Costo de entrar	Costo acumulado
0	(1, 1)	0 (inicio)	0
1	(1, 2)	1	1
2	(1, 3)	4	5
3	(1, 4)	1	6
4	(1, 5)	2	8
5	(1, 6)	1	9
6	(1, 7)	1	10
7	(2, 7)	2	12
8	(2, 8)	1	13
9	(2, 9)	1	14
10	(3, 9)	1	15
11	(4, 9)	2	17
12	(5, 9)	1	18
13	(6, 9)	1	19
14	(7, 9)	1	20
15	(8, 9)	3	23
16	(9, 9)	1	24
17	(9, 10)	1	25
18	(10, 10)	1	26

El programa termina con:

```
pasos: 18
costo total: 26
```

No existe otro camino con un costo menor que 26. Se comprobó con un algoritmo distinto (Dijkstra) en las pruebas automáticas.

6. Pruebas automáticas

El proyecto incluye `LaberintoTest.java`, un programa de pruebas que no necesita librerías externas. Para ejecutarlo, desde la carpeta del proyecto:

```
javac -d out Laberinto.java LaberintoTest.java
java -cp out LaberintoTest
```

Qué comprueba:

Prueba	Qué verifica
Mapa vs README	Que cada celda de `MAPA` coincide con la tabla del README.
Funciones de validación	`dentroDelTablero`, `esTransitable`, `definirInicio` y `definirFin` con casillas válidas, bordes, obstáculos, fuera de rango y meta igual al inicio.
`costo` y `heuristica`	Valores conocidos de costo y de distancia Manhattan.
Todos los pares	Para los 5550 pares de inicio y meta posibles: que la ruta empieza y termina donde debe, avanza de una en una, no pisa obstáculos y tiene el mismo costo que el óptimo calculado con Dijkstra.
Programa completo	Simula al usuario escribiendo en el teclado, con entradas válidas y con errores (texto, fuera de rango, obstáculo, meta = inicio).

Resultado actual: **5694 comprobaciones correctas y 0 fallidas**.

7. Observaciones

Detalles que conviene conocer. Ninguno afecta al uso normal del programa.

- **Entrada que se acaba antes de tiempo.** Si la entrada termina antes de dar las cuatro coordenadas (por ejemplo, al pasarle un archivo incompleto con `<`), `leerEntero` lanza `NoSuchElementException` y el programa se cierra con un error. Escribiendo en la terminal no ocurre.
- **Eficiencia.** `buscarMenorF` y `buscarNodo` recorren las listas completas en cada llamada. Para un tablero de 10 × 10 es instantáneo; en mapas muy grandes convendría usar una cola de prioridad (`PriorityQueue`) y un arreglo de visitados.
- **Empates.** Cuando hay varias rutas con el mismo costo mínimo, el programa elige una según el orden en que se revisan las vecinas (arriba, abajo, izquierda, derecha). Cualquiera de ellas es correcta.